

```
1 #include <errno.h>
2 #include <sys/types.h>
3 #include <sys/ipc.h>
4 #include <sys/sem.h>
5 #include <sys/msg.h>
6 #include <sys/wait.h>
7 #include <unistd.h>
8 #include <stdlib.h>
9 #include <stdio.h>
10
11 #define CNT_PROC 3
12
13 union semun {
14     int val;
15     struct semid_ds *buf;
16     unsigned short *array;
17 };
18
19 int fd[2];
20
21 int initsem(key_t semkey){
22     union semun semunarg;
23     int status = 0, semid;
24     semid = semget(semkey, 1, IPC_CREAT | IPC_EXCL | 0600);
25
26     if(semid == -1){
27         if(errno == EEXIST)
28             semid = semget(semkey, 1, 0);
29     }
30     else{
31         semunarg.val = 1;
32         status = semctl(semid, 0, SETVAL, semunarg);
33     }
34
35     if(semid == -1 || status == -1){
36         perror("initsem");
37         return(-1);
38     }
39
40     return semid;
41 }
42
43 int uninitsem(int *semid){
44     semctl(*semid, 0, IPC_RMID);
45 }
46
47 int semlock(int semid){
48     struct sembuf buf;
49     buf.sem_num = 0;
50     buf.sem_op = -1;
51     buf.sem_flg = SEM_UNDO;
52     if(semop(semid, &buf, 1) == -1){
53         perror("semlock failed");
54         exit(1);
55     }
56     return 0;
57 }
58
59 int semunlock(int semid){
60     struct sembuf buf;
61     buf.sem_num = 0;
62     buf.sem_op = 1;
63     buf.sem_flg = SEM_UNDO;
64     if(semop(semid, &buf, 1) == -1){
65         perror("semunlock failed");
66         exit(1);
67     }
68     return 0;
69 }
70
71 void semhandle(int *mqid, int *semid){
72     pid_t pid = getpid();
73     int len;
74
75     if((*semid = initsem(1)) < 0)
76         exit(1);
77
78     close(fd[1]);
79     len = read(fd[0], mqid, 4);
80     close(fd[0]);
81
82     msgsnd(*mqid, (int*)semid, 4, 0);
83
84     semlock(*semid);
85     printf("Lock : Process %d\n", (int)pid);
86     printf("!!! critical section\n");
87     sleep(1);
88     printf("Unlock: Process %d\n", (int)pid);
89     semunlock(*semid);
90
91     exit(0);
92 }
93
94 int main(void){
95     int a;
96     int semid = 0;
97     int mqid = 0;
98     pid_t pid = getpid();
99
100     mqid = msgget(10, IPC_CREAT|0666);
101
102     if(pipe(fd) == -1){
103         perror("pipe");
104         exit(1);
105     }
106
107     printf("parent pid:%d\n", (int)pid);
108     for(a = 0; a < CNT_PROC; a++){
109         if(fork() == 0) semhandle(&mqid, &semid);
110
111     }
112     close(fd[0]);
113     write(fd[1], (int*)&mqid, 4);
114     close(fd[1]);
115
116     msgrcv(mqid, (int*)&semid, 4, 0, 0);
117
118     for(a = 0; a < CNT_PROC; a++){
119         pid_t ch_pid = waitpid(-1, 0, 0);
120     }
121
122     uninitsem(&semid);
123
124     msgctl(mqid, IPC_RMID, 0);
125
126     return 0;
127 }
```